```
            .style("text-anchor", "end")
            .attr("transform", "translate(-10,-10) rotate(90)");
    };
};
```

Rotates the text on the y-axis

A few new things are going on here. For one, we're using a new scale: `d3.scale.ordinal`, which takes an array of distinct values and allows us to place them on an axis like we do with the names of our nodes in this example. We need to use a scale function that you haven't seen before, `rangePoints`, which creates a set of bins for each of our values for display on an axis or otherwise. It does this by associating each of those unique values with a numerical position within the range given. Each point can also have an offset declared in the second, optional variable. The other new piece of code uses queue.js, which we need because we're loading two CSV files and we don't want to run our function until those two CSVs are loaded. We're building this matrix array of objects that may seem obscure. But if you examine it in your console, you'll see, as in figure 6.2, it's just a list of every possible connection and the strength of that connection, if it exists.

Figure 6.3 shows the resulting adjacency matrix based on the node list and edge list.

You'll notice in many adjacency matrices that the square indicating the connection from a node to itself is always filled. In network parlance this is a *self-loop*, and it occurs when a node is connected to itself. In our case, it would mean that someone



Figure 6.2 The array of connections we're building. Notice that every possible connection is stored in the array. Only those connections that exist in our dataset have a weight value other than 0. Notice, also, that our CSV import creates the weight value as a string.